

LESSON: SQL OPERATORS

I. Learning Objectives

After this lesson, students should be able to:

1. Explain what an **operator** is in SQL.
2. Identify the different types of SQL operators.
3. Use operators in SELECT, WHERE, and other SQL statements.
4. Combine operators to create more specific queries.
5. Apply SQL operators to real-world database scenarios.

II. What is an Operator in SQL?

An **operator** is a symbol or keyword used in SQL to perform an **operation or comparison**.

Simple Definition

An operator is used to compare, calculate, or combine conditions in SQL.

For example:

```
SELECT *
```

```
FROM Students
```

```
WHERE Age > 18;
```

The > symbol is a **comparison operator**.

It means:

"Display the students whose age is greater than 18."

So, when SQL sees:

```
Age > 18
```

it checks whether the condition is **TRUE or FALSE**.

III. Main Types of SQL Operators

In SQL Server, commonly used operators can be grouped into the following types:

1. **Arithmetic Operators**
2. **Comparison Operators**
3. **Logical Operators**
4. **Bitwise Operators**
5. **Special Operators**

For beginners, the most important operators to understand first are:

- **Arithmetic**
- **Comparison**
- **Logical**

We will also look at several **special operators**, such as BETWEEN, IN, LIKE, and IS NULL.

1. Arithmetic Operators

Arithmetic operators are used to perform **mathematical calculations** in SQL.

Operator	Meaning
----------	---------

+	Addition
---	----------

-	Subtraction
---	-------------

*	Multiplication
---	----------------

/	Division
---	----------

%	Modulo or remainder
---	---------------------

Example Table: Products

ProductID	ProductName	Price	Quantity
1	Keyboard	500	2
2	Mouse	300	3

3 Monitor 5,000 2

Example 1: Addition

SELECT 10 + 5 AS Result;

Result

Result

t

15

SQL performs the calculation:

10 + 5 = 15

Example 2: Multiplication

Suppose we want to calculate the total amount for each product.

SELECT

 ProductName,

 Price,

 Quantity,

 Price * Quantity AS TotalAmount

FROM Products;

Result

Product	Price	Quantity	TotalAmount
Keyboard	500	2	1,000
Mouse	300	3	900
Monitor	5,000	2	10,000

For example:

Price = 500

Quantity = 2

$$500 \times 2 = 1,000$$

Therefore:

Price * Quantity

calculates the **total amount**.

Key idea

Arithmetic operators perform calculations.

2. Comparison Operators

Comparison operators are especially important when using the **WHERE clause**.

They are used to compare values and determine whether a condition is **TRUE or FALSE**.

Operator	Meaning
=	Equal to
<>	Not equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example Table: Students

StudentID	Name	Age	Course
1	Juan	20	BSIS
2	Maria	18	BSIT
3	Pedro	22	BSCS
4	Ana	19	BSIS

A. Equal To =

```
SELECT *  
FROM Students  
WHERE Course = 'BSIS';
```

Meaning

Display all students whose course is **BSIS**.

Result

StudentID	Name	Age	Course
1	Juan	20	BSIS
4	Ana	19	BSIS

The condition is:

Course = 'BSIS'

SQL checks each row and keeps the rows where the condition is TRUE.

B. Greater Than >

```
SELECT *
```

FROM Students

WHERE Age > 19;

Meaning

Display students whose age is **greater than 19**.

Result

Name	Age
------	-----

Juan	20
------	----

Pedro	22
-------	----

Important:

Age > 19

means **more than 19**.

Therefore, a student who is exactly 19 is **not included**.

C. Less Than <

SELECT *

FROM Students

WHERE Age < 20;

Meaning

Display students who are **younger than 20**.

Result

Name	Age
------	-----

Maria	18
-------	----

Ana	19
-----	----

D. Greater Than or Equal To >=

```
SELECT *  
FROM Students  
WHERE Age >= 20;
```

Meaning

Display students who are **20 years old or older**.

Result

Name	Age
------	-----

Juan	20
------	----

Pedro	22
-------	----

Important Difference

Compare:

Age > 20

and:

Age >= 20

> means:

More than 20

While >= means:

20 or more

Therefore, a student who is exactly 20 is included when we use >= 20.

3. Logical Operators

Logical operators are used when we have **more than one condition**.

The three most common logical operators are:

Operator Meaning

r

AND	Both conditions must be TRUE
OR	At least one condition must be TRUE
NOT	Reverses the condition

A. AND Operator

AND means:

Both conditions must be TRUE.

Example:

SELECT *

FROM Students

WHERE Course = 'BSIS'

AND Age >= 20;

The student must satisfy **both** conditions:

Course = 'BSIS' → TRUE

Age >= 20 → TRUE

Only when both are TRUE will the record be included.

Truth Logic

Condition 1	Condition 2	AND
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

Easy way to remember:

AND = BOTH must be TRUE.

B. OR Operator

OR means:

At least one condition must be TRUE.

Example:

SELECT *

FROM Students

WHERE Course = 'BSIS'

OR Course = 'BSIT';

This means:

Display students who are taking **BSIS or BSIT**.

The two conditions do not both have to be TRUE.

Truth Logic

Condition 1	Condition 2	OR
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

Easy way to remember:

OR = AT LEAST ONE must be TRUE.

C. NOT Operator

NOT reverses a condition.

Example:

```
SELECT *
```

```
FROM Students
```

```
WHERE NOT Course = 'BSIS';
```

This means:

Display all students whose course is **not BSIS**.

Conceptually:

```
NOT Course = 'BSIS'
```

is equivalent to:

```
Course != 'BSIS'
```

Easy way to remember:

NOT = reverse the condition.

4. BETWEEN Operator

BETWEEN is used when we want to find values **within a specific range**.

Example:

```
SELECT *
```

```
FROM Students
```

```
WHERE Age BETWEEN 18 AND 20;
```

This means:

Display students whose age is from **18 to 20**.

The important thing to remember is:

BETWEEN is inclusive.

Therefore, these values are included:

18

19

Easy way to remember:

BETWEEN = within a range, including the boundaries.

5. IN Operator

The IN operator is useful when a column can match **multiple possible values**.

Without IN, we could write:

```
SELECT *
```

```
FROM Students
```

```
WHERE Course = 'BSIS'
```

```
OR Course = 'BSIT'
```

```
OR Course = 'BSCS';
```

But we can write the same idea more clearly using IN:

```
SELECT *
```

```
FROM Students
```

```
WHERE Course IN ('BSIS', 'BSIT', 'BSCS');
```

The meaning is:

"Is the Course one of these listed values?"

So:

```
IN ('BSIS', 'BSIT', 'BSCS')
```

means the course can be:

- BSIS
- BSIT
- BSCS

Easy way to remember:

IN = matches one of the values in the list.

6. LIKE Operator

The LIKE operator is used to search for a **text pattern**.

Example:

```
SELECT *
```

```
FROM Students
```

```
WHERE Name LIKE 'J%';
```

This means:

Display names that **start with the letter J**.

Possible matches include:

Juan

John

James

Joseph

Important LIKE Symbols

Symbols	Meaning
%	Any number of characters
_	Exactly one character

Example 1

```
WHERE Name LIKE 'J%'
```

Means:

The name starts with **J**.

Examples:

Juan

John

James

Example 2

WHERE Name LIKE '%a'

Means:

The name ends with **a**.

Example 3

WHERE Name LIKE '%an%'

Means:

The name contains **an** somewhere in the text.

Example 4

WHERE Name LIKE '_uan'

Means:

There is exactly **one character before uan**.

For example:

Juan

The **_** represents the letter J.

Easy way to remember:

% = many characters

_ = exactly one character

7. IS NULL Operator

This is very important.

In SQL, NULL does **not** mean zero.

It also does not mean an empty string.

NULL means:

There is no stored value, or the value is unknown.

For example, to find students who do not have an email value:

```
SELECT *
```

```
FROM Students
```

```
WHERE Email IS NULL;
```

This means:

Display students whose Email value is NULL.

Do NOT use:

```
WHERE Email = NULL;
```

Use:

```
WHERE Email IS NULL;
```

If we want students who **have an email value**, we use:

```
WHERE Email IS NOT NULL;
```

Easy way to remember:

Use IS NULL to check for NULL values.

8. Modulo %

The % operator returns the **remainder after division**.

Example:

```
SELECT 10 % 3 AS Remainder;
```

Result

Remainder

1

Why?

$10 \div 3 = 3$ remainder 1

Therefore:

$10 \% 3 = 1$

Practical Example: Finding Even and Odd Numbers

Suppose we have a StudentID.

To find students with **even StudentIDs**:

```
SELECT *
```

```
FROM Students
```

```
WHERE StudentID % 2 = 0;
```

Why does this work?

When a number is divided by 2:

- remainder 0 → even
- remainder 1 → odd

For odd StudentIDs:

```
SELECT *
```

```
FROM Students
```

```
WHERE StudentID % 2 <> 0;
```

Easy way to remember:

% gives you the remainder.

9. Combining Operators

In real-world SQL queries, we often need to combine several operators.

Example

Suppose the question is:

"Find BSIS students who are between 18 and 22 years old."

We can write:

```
SELECT *
```

```
FROM Students
```

```
WHERE Course = 'BSIS'
```

```
AND Age BETWEEN 18 AND 22;
```

Here we are using three concepts:

= → Comparison

AND → Logical

BETWEEN → Range

The query means:

Find students whose course is BSIS **AND** whose age is between 18 and 22.

Another Example

Question:

"Find students who are taking BSIS or BSIT and are at least 18 years old."

SQL:

```
SELECT *
```

```
FROM Students
```

```
WHERE Course IN ('BSIS', 'BSIT')
```

```
AND Age >= 18;
```

Breakdown

Course IN ('BSIS', 'BSIT')

means:

BSIS OR BSIT

And:

Age >= 18

means:

18 years old or older.

The AND requires both parts to be satisfied.

10. Operator Precedence

When a query contains multiple operators, SQL does **not always simply evaluate everything from left to right**.

For example:

```
SELECT *
```

```
FROM Students
```

```
WHERE Course = 'BSIS'
```

```
OR Course = 'BSIT'
```

```
AND Age >= 20;
```

The AND condition is evaluated before the OR condition.

To make the logic clearer, use **parentheses**:

```
SELECT *
```

```
FROM Students
```

```
WHERE (Course = 'BSIS' OR Course = 'BSIT')
```

```
AND Age >= 20;
```

Now the meaning is very clear:

Step 1

The student must be:

BSIS OR BSIT

Step 2

The student must also be:

20 years old or older

Professor's Tip 🗨️

When a SQL condition becomes complex, use parentheses () to make the intended logic clear.

11. Complete Practical Example

Let's imagine we have an Employees table.

EmployeeID	EmployeeName	Department	Salary	Age
1	Juan	IT	30,000	25
2	Maria	HR	28,000	30
3	Pedro	IT	40,000	35
4	Ana	Finance	35,000	28

Question

Find IT employees whose salary is greater than 30,000.

SQL:

```
SELECT EmployeeName, Department, Salary
```

```
FROM Employees
```

```
WHERE Department = 'IT'
```

```
AND Salary > 30000;
```

Result

EmployeeName	Department	Salary
Pedro	IT	40,000

Why is Juan not included?

Juan:

Department = IT → TRUE

Salary > 30,000 → FALSE

Because we used AND, **both conditions must be TRUE.**

Therefore:

TRUE AND FALSE = FALSE

Juan is not included.

Why is Pedro included?

Pedro:

Department = IT → TRUE

Salary > 30,000 → TRUE

Therefore:

TRUE AND TRUE = TRUE

Pedro is included in the result.

12. SQL Operators Summary

Operator	Type	Example	Meaning
+	Arithmetic	Price + Tax	Add
-	Arithmetic	Price - Discount	Subtract
*	Arithmetic	Price * Quantity	Multiply
/	Arithmetic	Salary / 12	Divide
%	Arithmetic	ID % 2	Remainder
=	Comparison	Age = 18	Equal

<>	Comparison	Age <> 18	Not equal
!=	Comparison	Age != 18	Not equal
>	Comparison	Age > 18	Greater than
<	Comparison	Age < 18	Less than
>=	Comparison	Age >= 18	Greater than or equal
<=	Comparison	Age <= 18	Less than or equal
AND	Logical	Age > 18 AND Course='BSIS'	Both conditions
OR	Logical	Course='BSIS' OR Course='BSIT'	Either condition
NOT	Logical	NOT Course='BSIS'	Reverses condition
BETWEEN	Special	Age BETWEEN 18 AND 25	Within a range
IN	Special	Course IN ('BSIS','BSIT')	Matches listed values
LIKE	Special	Name LIKE 'J%'	Pattern matching
IS NULL	Special	Email IS NULL	No stored value

Professor's Key Takeaway

Class, when writing a SQL query, think about these three questions:

1. WHAT do I want to display?

Use:

SELECT



2. WHERE should I get the data from?

Use:

FROM



3. WHAT conditions should the data follow?

Use:

WHERE + OPERATORS

For example:

```
SELECT EmployeeName, Salary
```

```
FROM Employees
```

```
WHERE Department = 'IT'
```

```
AND Salary >= 30000;
```

Read it as a sentence:

"Select the employee name and salary from the Employees table where the department is IT and the salary is 30,000 or higher."

That is the basic mindset you should develop when learning SQL.

Do not simply memorize SQL operators. Understand what condition you want the database to check or what calculation you want it to perform.

Quick Memory Guide

Arithmetic → Calculate

+ - * / %

Comparison → Compare

= != <> > < >= <=

Logical → Combine conditions

AND OR NOT

Special → Search or filter in specific ways

BETWEEN IN LIKE IS NULL